CO1035: Game Concepts
Lab 4A – Direction Vectors

Introduction

- In this lab, you are going to carry on working with vectors and more specifically with direction vectors.
- Before you start working on this lab's tasks, use last week's materials to refresh your knowledge of direction vectors
- Now that you have refreshed your knowledge of vectors, let's get on to the lab. The project you will be creating this week will make use of direction vectors and the length of vectors

Setting Up

1. Create a TL-Engine project called **Lab4A_Drones_Project**.
2. Download the associated media files for the project from Blackboard, and save them to a folder called **media** inside your project folder. Now use the **AddMediaFolder()** method to add the folder's path to your project.
3. Use the "**ground.x**" mesh to create a floor model at the origin. Then scale the floor model so that it is twice its initial size.

```
floor->Scale(2);
```

4. Use the "**drone.x**" mesh to create two drone models named "**playerDrone**" and "**enemyDrone**".

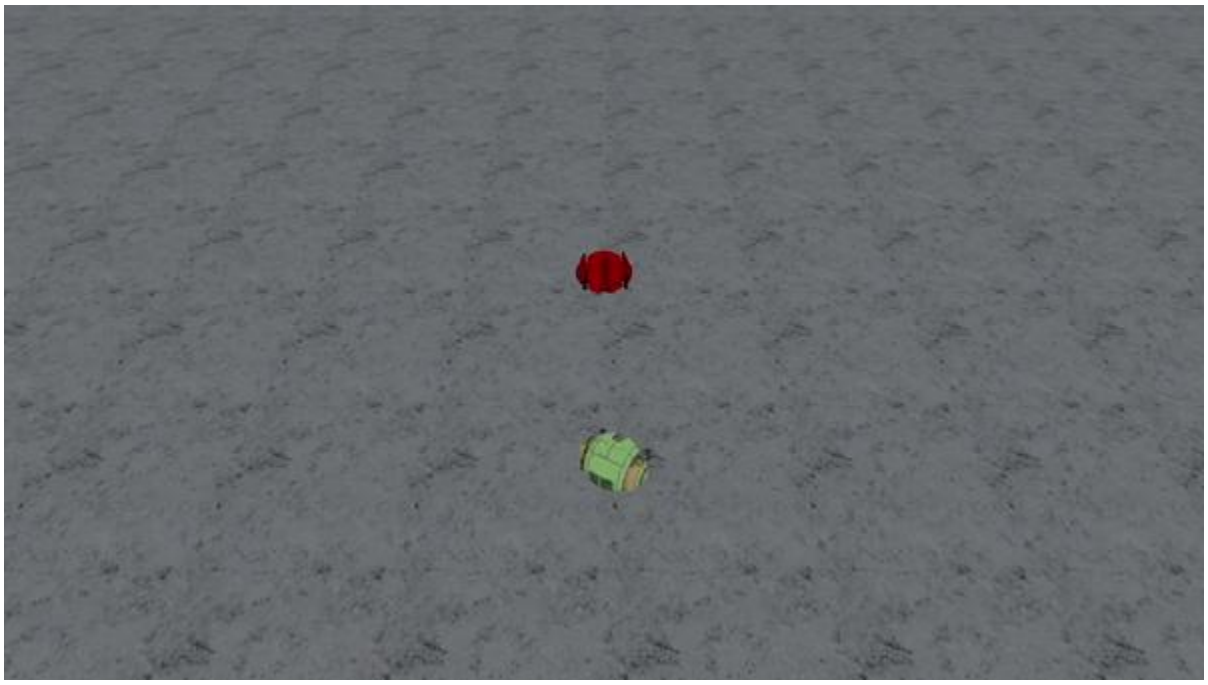
    - Position **playerDrone** at (0.0f, 60.0f, 0.0f),
    - Position **enemyDrone** at (0.0f, 60.0f, 400.0f)
    - Set the skin of **enemyDrone** to "**Red_Drone.jpg**".

5. Create a **kManual** camera and **position** it at (0.0f, 600.0f, -250.0f) and **rotate** it to **45 degrees** about its **local X-axis** so that it is tilted down.
6. Declare an **IMesh*** variable named "**bulletMesh**" and load the "**bullet.x**" mesh. Then declare an **IModel*** variable named "bullet", but don't initialize it yet (it will be used later).
7. Create a constant float called **kGameSpeed** and initialize it to 0.05f (reduce or increase this value if it is too fast or too slow on your machine).

8. In your game loop, add code that responds to holding the W key to move **playerDrone** forward along its **local z-axis**, and responds to holding A and D keys by rotating **playerDrone** left and right along its **local y-axis**. Use **kDroneSpeed** to determine the speed of these movements e.g., you can move **playerDrone** forward at a speed equivalent to **kGameSpeed** and rotate it at a speed equivalent to **kGameSpeed/2**.

9. **LookAt()** is a method for models (or cameras) that **points the local Z-axis towards another model**. This effectively makes the first model/camera look at the second; see an example of the method's use below (do not copy this code into your project). Use the new '**LookAt'** method to make **enemyDrone** always point at **playerDrone**. Think carefully about whether to put this in the initialization section or the game loop. Move **playerDrone** around to test that **enemyDrone** continues to point at it.

```
shooter->LookAt(target);
```



Getting the Vector between two models

1. For **enemyDrone** to fire a bullet at **playerDrone**, the direction of movement between **playerDrone** and **enemyDrone** must be known. **The direction of movement between two points is described by a vector**. The vector between the two points where the two models are positioned can be calculated by **subtracting the position of one point from the position of the second point**. An example of how this can be done is shown below (do not copy this code). **Note:** The order in which you perform the subtraction above **will affect the direction of your vector**, since the direction of the vector from Model1 to Model2 is opposite to that of the vector from Model2 to Model1.

```
float vectorX;
```

```
float vectorY;
float vectorZ;
vectorX = Model2->GetX() - Model1->GetX();
vectorY = Model2->GetY() - Model1->GetY();
vectorZ = Model2->GetZ() - Model1->GetZ();
```

2. In the game loop, write code to calculate the vector from **enemyDrone** to **playerDrone**. The x, y, and z coordinates of **enemyDrone** and **playerDrone** can be found using **GetX(), GetY(),** and **GetZ()** methods. The result of the calculation for each component of the vector should be assigned to **vectorX**, **vectorY,** and **vectorZ** as shown in the code snippet above.

Length of a Vector (Distance Between Two Points)

1. The distance between two points is the length of the vector between the two points. The length of the vector is given by:

$$\|V\| = \sqrt{x^2 + y^2 + z^2}$$

2. Declare three new floats named **distanceX**, **distanceY,** and **distanceZ** for storing the squares of the vector components you calculated in the last section, i.e., **vectorX**, **vectorY,** and **vectorZ**. Use simple multiplication to derive the square of any two values, e.g.:

```
distanceX = vectorX * vectorX; // distanceX squared
```

3. Sum the squares of the components and then calculate the square root of the result to get the length of the vector. There is a function within C++ to calculate square roots called **sqrt()**. To use it, you need to add the following two lines of code to the top of your program.

```
#include <iostream>
using namespace std;
```

This will give you access to the C++ math library and the **sqrt()** function. The function takes just one parameter, e.g.:

```
float root;
root = sqrt(9*9); // the square root of 9 squared
```

Now use the **sqrt()** method to calculate the distance between **enemyDrone** and **playerDrone** (the length of the vector). Use a new float variable called "distance" to store the distance calculated.

The Direction Vector

1. You have derived the vector between the **enemyDrone** and **playerDrone**. You have also calculated the length of the vector. To use these values to move a bullet fired by **enemyDrone**

towards **playerDrone** you are going to have to calculate the direction vector between the two drones; therefore, you have to **normalize** the vector you have derived.

2. The direction vector is independent of the distance between two points. For example, it allows you to move a model in a particular direction, disregarding the speed of the movement. If the vector between the two positions is **v** (**x, y, z**), and the length of this vector is "**len**", then the direction vector is (**x/len, y/len, z/len**)

3. To calculate the direction vector from **enemyDrone** to **playerDrone**, first create three new float variables: **directionX**, **directionY,** and **directionZ**. Then use the variables to store the components of your direction vector as shown below:

```
//sum of the squares of the vector's components
float distance = distanceX + distanceY + distanceZ;
distance = sqrt( distance ); // the square root of distance
float directionX = vectorX / distance;
float directionY = vectorY / distance;
float directionZ = vectorZ / distance;
```

4. Check that you are dividing each vector component by the distance and not, for example, **distanceX** by distance.

5. Next, create a new Boolean variable (in the initialization section) called '**bulletShot'** and initialize it to false. You will use this variable to check whether a bullet has been fired by **enemyDrone**.

   - Check if the distance between **enemyDrone** and **playerDrone** is less than 250 (use your float variable "**distance**") and **bulletShot** is false.

   - If both these conditions are true, then:

     - Using the **IModel*** variable named bullet you declared in the initialization section, create a bullet model using **bulletMesh**. Create it in the same position as **enemyDrone**. **Note that you are creating the model inside the game loop**.
     - Initialize the bullet direction vector variables using **directionX**, **directionY,** and **directionZ** from above.
     - Set **bulletShot** to true, so that another bullet is not created.

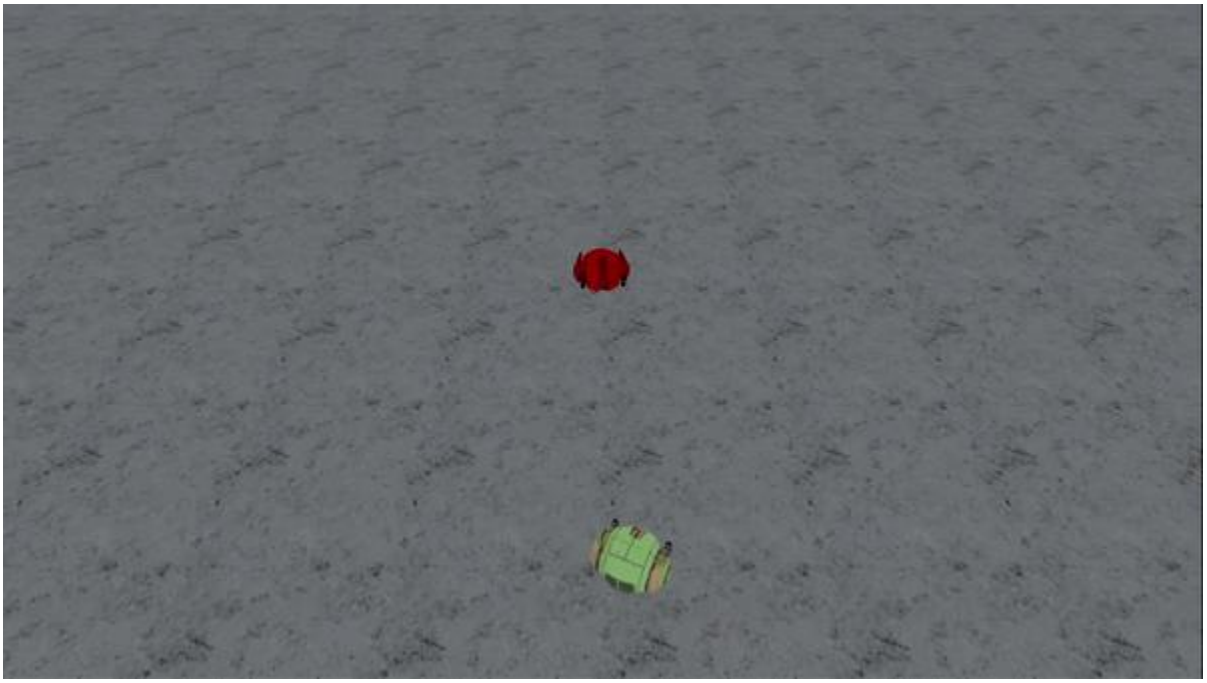
6. Declare a constant called **kBulletSpeed** in the initialization section and set its value to "**kGameSpeed * 3**". Then add another 'if' statement to test if the '**bulletShot'** variable is true; if so, then move the bullet model by the bullet direction vector scaled by **kBulletSpeed**.

```
bullet->MoveX( directionX * kBulletSpeed );
bullet->MoveY( directionY * kBulletSpeed );
bullet->MoveZ( directionZ * kBulletSpeed );
```

7. Finally, inside the "if" statement created in 6 above, calculate the distance between the bullet and **enemyDrone**, i.e., the length of the vector between the bullet and the **enemyDrone**, if it is greater than 150, then remove the bullet model and set bulletShot to false. This will allow

bullets to disappear after travelling a certain distance and allow the enemy Drone to fire new bullets.

8. You should now have an enemy drone that tracks your drone and shoots at it when it comes near. Read the code you have written, paying special attention to the logic that is used to create, move, and destroy the bullets.



Introducing Variable Timing

1. You may have noticed that the speed of the objects in your game reduces when you introduce additional models or perform complex operations every frame. You may have also noticed that the same program runs at different speeds when executed on different machines, e.g., the games lab PC vs your PC. This is because so far, you have been moving models every frame by a given amount of units, and the number of frames per second rate of your program depends on the type of machine you have as well as the complexity of the program. In this section, you will learn to create consistent programs using variable timing.

2. The first step is to initialize the timer by adding the code below in the initialization section of your program.

```
myEngine->Timer();
```

3. You then need to call **Timer()** once per frame, at the beginning of the game loop, immediately after `myEngine->DrawScene();`. Declare a variable of type float called **frameTime** and assign the value returned by **Timer()** to it; this value represents the time taken in **ms** for the last scene to be rendered.

```
float frameTime = myEngine->Timer();
```

4. Next, change the value of **kGameSpeed** to 50; this should automatically update the constants initialized using **kGameSpeed,** i.e. **kDroneSpeed** and **kBulletSpeed**. Then update your game loop so that the speed used by all the move and rotate methods is multiplied by **frameTime**.

```
if (myEngine->KeyHeld(Key_W))
{
    playerDrone->MoveLocalZ(kDroneSpeed * frameTime);
}
if (myEngine->KeyHeld(Key_A))
{
    playerDrone->RotateLocalY(-kDroneSpeed * frameTime / 2);
}
else if (myEngine->KeyHeld(Key_D))
{
    playerDrone->RotateLocalY(kDroneSpeed* frameTime / 2);
}
```

This essentially means that your objects now have a velocity of units/second; for example, the **playerDrone** is now moving at a speed of 50 units/second regardless of the machine type or complexity of the program.

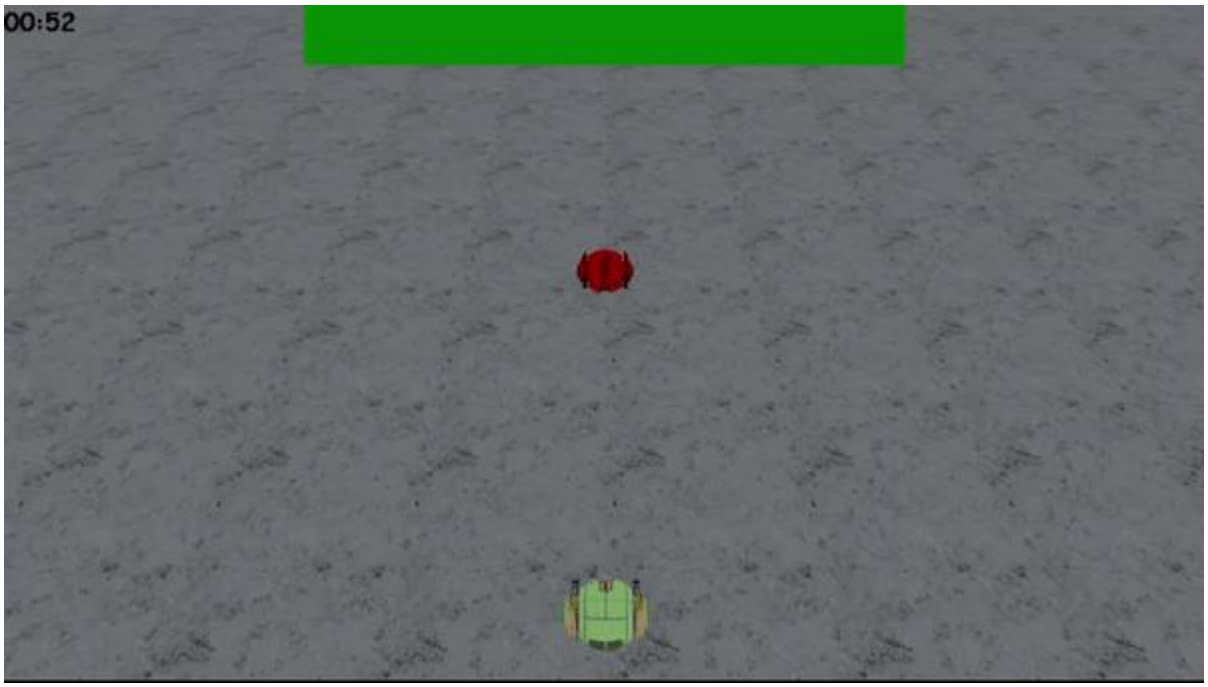
5. Make sure to update the bullet move methods as shown below:

```
bullet->MoveX( directionX * kBulletSpeed * frameTime );
bullet->MoveY( directionY * kBulletSpeed * frameTime );
bullet->MoveZ( directionZ * kBulletSpeed * frameTime );
```

Extending the Drones Prototype

1. If you have completed the main lab tasks, here are a few ideas that you can use to extend your program:

- Test if the bullets have hit **playerDrone** (use vector distance to check if the bullet is very close to the player).
- Keep track of the **playerDrone's** health using a health variable. The variable should be initialized as 100 at the start of the game, and each bullet hit should reduce its value by 10 points. Use the **Lab4A – Healthbars (Extended Lab).zip** folder from Blackboard to create a health bar centered at the top of the screen that shows the value of the health variable. A single sprite created using "**greenbox.png**" should be used to represent 10 units of health; therefore, at the start of the game, 10 green boxes should be centered at the top of the screen. Every time the player gets hit by a bullet, a green box should be replaced by a red box starting from the rightmost box.
- Display the time that has passed since the program was started in the format **MM:SS** at the top left corner of the screen. **Hint: Timer()** method returns the time that has passed since the last frame was rendered (in milliseconds).


00:52